

1.

Task Tracker - Bug Fixes

Tech - React + Spring Boot + H2

Priority of bugs

1. SQL AND/OR bug

→ wrong data was shown

2. Thread sleep

→ Unnecessary 1 sec delay

3. useTasks hook

→ Race condition + loading + API calls

4. In-memory pagination

→ Does not scale

5. Input validation

→ Invalid input caused error

6. Page reset → Search showed "No Tasks" found

why this order → wrong data > slow response
so I fixed SQL bug

(2)

Then &

Performance → Reliability →
Scalability → & I

Bug 1 SQL AND/OR Precedence

Where? → TaskRepository.java

→ SearchTasks.java

→ tasks-search-package = SQL

How I find?

empty search was returning → 49 tasks

But 2 → were archived

expected → 47 tasks

Root Cause → In SQL

AND has higher priority than OR

So the query was interpreted as
(archived = false AND title LIKE term)

OR

(description LIKE term AND status filter)

③

Problems

1. Archived tasks could appear
2. Status filter could be ignored.

fix → archived = FALSE
AND (title OR description)
AND status filter

Result → 49 → 47

→ status filtering also works correctly.

~~Bug 1~~

Bug 2 - Artificial Delay
where

→ TaskController.java

How I found it?

→ small/empty search was taking too much time

The code said it was for complexity estimation / logging, but it was actually sleeping

Root cause → shorter query → higher score → more sleep

(4)

for empty query

$$100 \times 100 \text{ ms} = 10000 \text{ ms} = 10 \text{ sec}$$

→ so every req had unnecessary latency

Extra problem

→ Frontend was making API calls on every keystroke

example

a → API

ab → API

api → API

so typing also felt slow

Fix → Removed the complete Thread.sleep block.

kept only logging

log.info(...)

Result → Response time

~1 sec → ~3 sec

Main learning

→ logging should not introduce artificial latency

⑤ Bug 3 - useTasks Hook

where → frontend/src/hooks/useTasks.js

There were 4 issues

1. Race condition

→ uses types: $a \rightarrow ap \rightarrow api$

Suppose

'a' → request slow

'api' → req → fast

api response comes first

Then old a response comes later

→ can override the latest res

fix - use → cancelled flag + cleanup fn
→ old req result is ignored

2. Loading Gets Stuck

original flow:

• then() → setLoading(false)

• catch() → error

if req fails → then() does not run
so loading = true

②

and spinners can remain

fix → use → `finally()`

↳ So loading is stopped after both success and failure

3. Error not cleared
if one req fails

req 1 → Error

Then next req succeeds

req 2 → Success

old error can still remain.

fix → At the beginning of every req
`setError(null)`

4. API call on every keystroke
typing a → ap → api
means 3 API calls

fix → use 300 ms debounce

`setTimeout()`

+

`clearTimeout()`

Benefit → fewer API calls

↓

smooth UI

apsara

⑦ Bug 4 → In-memory pagination

where? TaskController.java

old approach

↳ repo returned all matching rows

Database

↓

All tasks

↓

list <Task>

↓

sublist()

↓

req Page

Problem

Suppose user needs only?

10 rows

But database returns

↳ 100,000 rows

All rows come into application memory first

Root Cause

↳ pagination was happening in java memory, not in DB.

Small data → okay

large data → slow + memory heavy.

for use → pageable + count query

Page < Task > searchTasks(} controller ?

... ,

pageable pageable

} PageRequest.of(Page 1, page size)

(8)

Now DB handles:

limit 10

offset 20

And separate COUNT(*)

→ return total count

New flow

Database



LIMIT + OFFSET



only required rows



Application

Extra fix

ordering &

ORDER BY created-at

ASC & id ASC

→ works as a tie-breaker

This keeps pagination
order stable when

time stamps are same.

otherwise rows can

repeat / disappear

Bug 5 - Input Validation

④

apsara

Date: _____

where $\rightarrow$ TaskController.java

Problem 1 - Invalid status
Task status value of ("bogus")

Caused:

$\rightarrow$ IllegalArgumentException

$\downarrow$

SDO error

But this is actually a client input problem
 $\rightarrow$ It should return $\rightarrow$ 400 Bad req

Problem-2 - Invalid page

If page = 0 Then $\rightarrow (0+1) \times 10 = 10$

so

$\rightarrow$ Sublist (-10, ...)

$\rightarrow$ causes a crash.

Problem-3 - No Page size limit

User could send:

pageSize = 10000000

There was no proper limit

$\rightarrow$ fix $\rightarrow$ use @RestControllerAdvice

invalid values return GlobalExceptionHandler

$\rightarrow$ 400 Bad req

Pageination Validation

$\rightarrow$ page = max(page, 1)

(10)

AND

$$1 \leq \text{page size} \leq 100$$

important concept

Yxx → client mistake

Sxx → Server mistake

→ so invalid user input should not normally become a 500 server error

Bug 6 - Page Reset

where → Render / sec / App.js

Problem → Suppose → current page = 3
Then user applies a search filter.

New filtered data has only:
1 page → But application still req-
page 3

→ So UI shows
→ No tasks found
even though matching tasks exist

(11)

fix → whenever search / filter changes

Reset page → 1

flow

Page 3
↓

Search / filter changes
↓

Reset page
↓

Page 1
↓

correct results

The board now uses the section's filter key. So page automatically resets to 1.

other improvements

→ These are not main bugs & but I also improved some areas

1. wildcard escaping

Problem → % & - can behave as SQL wildcards

fix Escape ' ' → so user can search from literal

(12)

2. Backend Cleanup
 Changed System.out
 to

Also

SLF4J Logger

- show-sql → off
- Open-in-view → off
- Added TaskResponse DTO

3. Database indexes

- Added indexes for
 'archive_id', 'created-at', 'id',
 'status',
 'assigned'

Biggest Risk

- current search uses
 LIKE '%search%' with LOWER(...)

Normal indexes may not work effectively

For large data → full table scan
 → Can happen.

Future Solution

Full text search